

Caporale Barbería

PLANIFICACIÓN DE SPRINT (SCRUM — DURACIÓN: 2 SEMANAS)

Este documento contiene el estado actual del desarrollo, la arquitectura técnica de base y el **Sprint Backlog** estructurado para las próximas dos semanas. Su objetivo es alinear el desarrollo de las nuevas funcionalidades (turnos recurrentes, dashboards, facturación y seguridad avanzada).

1. Stack Tecnológico y Estado Actual

- **Backend:** ASP.NET Core MVC / Web API (C#), Entity Framework Core.
- **Base de Datos:** SQLite (Maneja un índice único compuesto: UNIQUE(HairdresserId, Date)).
- **Frontend:** TypeScript (TS) modular en el cliente y vistas dinámicas con **Razor (.cshtml)**.
- **Estado del Módulo de Turnos:** Ya está implementada la lógica de cancelación asincrónica (*Soft Delete* mediante `IsCanceled = true`) y el **patrón de reciclado de registros** en el método `InsertAppointmentAsync` para reutilizar filas existentes y no romper la restricción UNIQUE de SQLite.

2. Objetivo del Sprint (Sprint Goal)

Consolidar el módulo de reservas con turnos recurrentes, desarrollar los paneles de control (Peluqueros y Administrador), implementar el flujo de caja/facturación y desplegar un sistema de baneo dual (Usuario y Hardware Fingerprinting) inter-navegador.

3. Sprint Backlog

Épica / Tarea	Descripción / Criterio Técnico
Épica 1: Gestión de Turnos Avanzada y Recurrentes	
Tarea 1.1: Cierre del módulo de reserva actual	Asegurar el flujo feliz de reserva y cancelación con actualización dinámica de la UI en la grilla de turnos.
Tarea 1.2: Implementación de Turnos Fijos (Recurrentes)	Modificar el DTO de reserva y la tabla Appointments para soportar recurrencia: <i>Sin recurrencia, Cada 1 semana, Cada 2 semanas</i> . Desarrollar la lógica en el backend para generar automáticamente bloqueos de agenda a futuro, respetando el patrón de reciclado si ya existen filas canceladas en esas fechas.
Épica 2: Dashboard y Vista del Peluquero	
Tarea 2.1: Panel de Agenda Interno	Crear una vista privada para los barberos donde puedan loguearse y visualizar su grilla de turnos diaria y semanal.
Tarea 2.2: Filtros de Visualización	Filtros por estado del turno (Confirmados, Cancelados) y ordenamiento cronológico.
Épica 3: Dashboard de Administrador, Facturación y Configuración	
Tarea 3.1: Módulo de Facturación y Caja	Desarrollar reportes de ingresos en el panel de administrador. Filtrar facturación total por rangos de fecha (Día, Semana, Mes) y por rendimiento individual de cada peluquero.
Tarea 3.2: Gestión Dinámica de Precios	Crear interfaz de administración para modificar los precios de los servicios de corte y barbería en tiempo real sin tocar código.
Tarea 3.3: Panel de Control de Usuarios	Vista global para ver el listado de clientes registrados y sus historiales de asistencia.
Épica 4: Sistema de Seguridad Perimetral y Manejo de Baneos	
Tarea 4.1: Baneo Lógico por Cuenta (UserId)	Agregar la bandera IsBanned a la tabla de usuarios. Si está activa, impedir el login y la reserva de turnos desde el backend.
Tarea 4.2: Baneo por Hardware Avanzado (Cross-Browser Fingerprinting)	Frontend (TypeScript): Integrar la librería @fingerprintjs/fingerprintjs (versión Open Source). Extraer los componentes WebGL (GPU), AudioContext, Hardware Concurrency y memoria RAM para generar un hash único. Backend (ASP.NET Core): Crear la tabla BannedHardwareProfiles. Validar en la inserción de turnos y bloquear con HTTP 403 si coincide.

4. Criterios de Aceptación Técnicos

1. **Model Binding Prolijo:** Todos los nuevos endpoints de los controladores deben mapear parámetros tipados de forma exacta con los requests del frontend (utilizando objetos anónimos en formato camelCase para las respuestas JSON hacia TypeScript).
2. **Validación de Modelos:** Uso obligatorio de ModelState.IsValid antes de invocar cualquier lógica de los servicios en las nuevas acciones de los dashboards.
3. **Prevención de Nulos:** Mantener el blindaje del backend interceptando consultas vacías con FirstOrDefaultAsync antes de alterar estados de entidades.